

# Guida per installare e avviare l'app su Windows e sul telefono

Questa guida non dà per scontato niente: nessuna riga di comando, nessun programma, nessun termine tecnico che non venga spiegato prima di usarlo. Se un passaggio non torna, fermati lì – il resto della guida presuppone che quel passaggio sia andato a buon fine.

**Cosa faremo:** installare quattro programmi sul computer (una volta sola), scaricare il progetto, scrivere una o più chiavi segrete in un file, installare un'app sul telefono, e poi due semplici comandi ogni volta che vuoi riaprirla – sul telefono vero, non solo nel browser.

## IN QUESTA GUIDA

- | Prima di iniziare                   | Provider e API key                |
|-------------------------------------|-----------------------------------|
| 1 Installare Node.js                | 8 Scrivere le chiavi              |
| 2 Installare Python                 | 9 Installare Expo Go sul telefono |
| 3 Installare uv                     | 10 Avviare il "cervello" dell'app |
| 4 Installare GitHub Desktop         | 11 Avviare l'app sul telefono     |
| 5 Scaricare il progetto             | Come si usa, in breve             |
| 6 Aprire il terminale nel progetto  | Le prossime volte                 |
| 7 Installare i "pezzi" del progetto |                                   |

## Prima di iniziare

Due cose da darti, **prima** che tu arrivi al Passo 8:

1. Un invito su GitHub come collaboratore del progetto (arriva una email da GitHub – vai cliccato "Accept invitation").
2. Una o più "chiavi" (stringhe di lettere e numeri), mandate in privato – per messaggio, non per email in chiaro se possibile. Non farci nulla finché non arrivi al Passo 8: tienile semplicemente da parte.

Ti servirà anche un **telefono Android**, per il Passo 9 – e che computer e telefono siano sulla **stessa rete Wi-Fi** quando arrivi al Passo 11 (stesso router: non funziona con i dati mobili del telefono, e non funziona se uno dei due è collegato a una VPN).

**Tre cose da sapere prima di provarla, per non perdere tempo a chiederti se è un bug:**

- Nessun modello è ancora stato interrogato davvero – questo primo giro è anche il primo collaudo reale, quindi aspettati che qualche prompt vada ritoccato dopo averlo visto rispondere.
- Il manichino 3D non è mai stato visto girare fuori da un browser: con questa guida lo provi per la prima volta su un telefono vero.
- Non c'è ancora nessun account online: tutto gira sul computer di chi ha avviato il "cervello" dell'app (Passo 10), e i capi non si salvano da un riavvio all'altro (lo ripetiamo più avanti, perché sorprende).

1

## Installare Node.js

Node.js è il programma che fa girare la parte "app" del progetto.

1. Vai su [nodejs.org/en/download](https://nodejs.org/en/download)
2. Il sito riconosce da solo che sei su Windows: clicca il bottone "Windows Installer"
3. Si scarica un file tipo `node-v22.x.x-x64.msi`. Aprilo (doppio clic)
4. Segui l'installazione cliccando Avanti su ogni schermata, poi Installa, poi Fine. Le impostazioni di default vanno bene, non c'è nulla da cambiare.

VERIFICA CHE ABBI FUNZIONATO

1. Premi il tasto Windows, digita `PowerShell`, premi Invio – si apre una finestra blu (o nera) con del testo: è il **terminale**, il posto dove scriveremo dei comandi per tutta la guida.

2. Scrivi (o incolla) questo e premi Invio:

```
node -v
```

3. Deve rispondere con qualcosa tipo `v22.14.0`. Se invece dice "node non è riconosciuto...", chiudi PowerShell, riaprilo, e riprova – a volte serve un riavvio della finestra dopo un'installazione.

2

## Installare Python

Python è il programma che fa girare la parte che parla con l'intelligenza artificiale.

1. Vai su [python.org/downloads](https://python.org/downloads)
2. Clicca il bottone giallo "Download Python 3.1x.x" (deve essere 3.12 o più recente – se il sito propone una versione più vecchia, cerca nella pagina "Looking for a specific release?" e scegli una 3.12.x o 3.13.x)
3. Apri il file scaricato.

**Attenzione, questo passaggio si salta facilmente e serve**

Nella prima schermata dell'installer, in basso, c'è una casella "Add python.exe to PATH". Spuntala prima di cliccare "Install Now".

5. Aspetta che finisca, poi "Close".

VERIFICA (IN UNA FINESTRA POWERSHELL NUOVA – CHIUDI QUELLA VECCHIA E RIAPRINE UNA)

```
python --version
```

Deve rispondere `Python 3.12.x` o simile. Se dice "non riconosciuto", quasi sempre è perché la casella di sopra non era spuntata: disinstalla Python da "Impostazioni → App", e reinstallalo rifacendo attenzione a quel passaggio.

3

## Installare uv

`uv` è lo strumento che scarica i "pezzi" di cui Python ha bisogno per questo progetto.

1. Apri PowerShell (tasto Windows → PowerShell → Invio)
2. Incolla questo comando e premi Invio:

```
powershell -ExecutionPolicy Bypass -c "irm  
https://astral.sh/uv/install.ps1 | iex"
```

3. Aspetta che finisca (qualche secondo), poi chiudi e riapri PowerShell.

#### VERIFICA

```
uv --version
```

Deve rispondere con qualcosa tipo `uv 0.11.x`.

## 4 Installare GitHub Desktop

Questo è il programma con cui scarichi il progetto, senza dover imparare i comandi di Git.

1. Vai su [desktop.github.com](https://desktop.github.com)
2. Scarica e installa (doppio clic, poi segue da solo)
3. Alla prima apertura, accedi con l'account GitHub con cui hai accettato l'invito al passo "Prima di iniziare"

## 5 Scaricare il progetto

1. Apri GitHub Desktop
2. In alto: File → Clone Repository
3. Nella finestra che si apre, vai sulla scheda "GitHub.com" — dovresti vedere il progetto (si chiama `wardrobe`) nell'elenco. Selezionalo.
4. In basso scegli dove salvarlo sul tuo computer (va benissimo lasciare quello proposto, es. `C:\Users\TuoNome\Documents\GitHub\wardrobe`) e ricorda questo percorso: ti servirà tra un attimo.
5. Clicca Clone.

**6**

## Aprire il terminale nella cartella del progetto

1. Apri Esplora file (l'icona della cartella gialla nella barra in basso) e vai nella cartella dove hai clonato il progetto (Passo 5.4)
2. Tieni premuto **Shift** e fai **clic con il tasto destro** dentro la cartella (non su un file, nello spazio vuoto)
3. Scegli "Apri finestra PowerShell qui" (o su Windows 11, semplicemente "Apri nel terminale")

Da qui in poi, ogni comando di questa guida va scritto in questa finestra (o in una nuova aperta nello stesso modo, nella stessa cartella).

**7**

## Installare i "pezzi" del progetto (una volta sola)

Nella finestra PowerShell aperta al Passo 6, scrivi ed esegui, uno alla volta (aspetta che ognuno finisca prima del successivo – il primo impiega qualche minuto):

```
npm install
```

```
npm run api:sync
```

Se durante `npm install` vedi righe gialle "warning" o messaggi grigi, va tutto bene: sono normali, non sono errori. Un errore vero è scritto in rosso e interrompe il comando.

### Provider, API key, e perché paghiamo fal.ai

Prima di scrivere le chiavi al prossimo passo, tre parole che tornano spesso – vale la pena capirle una volta sola.

**Provider** è l'azienda a cui "affittiamo" il modello di intelligenza artificiale: chi legge le foto e propone gli outfit non è nostro, è un servizio a pagamento a cui l'app si collega. Il progetto ne tiene tre intercambiabili: Anthropic (Claude, quello di riferimento), OpenAI (GPT), e Google (Gemini). Sono intercambiabili per costruzione – cambiare provider costa un file di codice,

non un mese di lavoro – proprio perché non vogliamo scoprire fra sei mesi di essere legati a uno solo.

**API key** è una password legata a un account che paga – il nostro, non il tuo – e su cui si accumula la spesa a consumo, un po' come un numero di carta di credito. Da qui le due regole che contano: non finisce mai in un file che va su GitHub (per questo vive in `.env`, che il progetto ignora apposta), e si scambia per un canale privato, mai per email in chiaro.

Il progetto chiede al modello due lavori distinti, e ogni chiave può servire per l'uno, per l'altro, o per entrambi: **leggere una foto** (capire tipo, colore, tessuto di un capo) e **proporre un outfit** (scegliere cosa abbinare e spiegare perché). Quale provider fa quale lavoro si sceglie dalla schermata Modelli in uso, di cui parliamo dopo il Passo 11.

**fal.ai** non è un provider di linguaggio: è un servizio a parte che serve solo a "staccare" il capo fotografato dallo sfondo, come nelle foto di un catalogo online. Non è indispensabile – se manca quella chiave l'app funziona lo stesso, le foto restano semplicemente con lo sfondo originale. Ma un capo già isolato è anche più facile da leggere per il modello che lo analizza, quindi quando c'è aiuta anche il primo lavoro.

## 8

### Scrivere le chiavi

1. In Esplora file, entra nella cartella `services`, poi `api` (quindi: `wardrobe\services\api`)
2. Trova il file `.env.example` (se non lo vedi, in alto in Esplora file clicca "Visualizza" → spunta "Estensioni nome file" e "Elementi nascosti")
3. Copialo (clic destro → Copia, poi clic destro nello spazio vuoto → Incolla)
4. Rinomina la copia (che si chiamerà tipo `.env.example - Copia`) in `.env` – attenzione: il nome deve essere esattamente `.env`, niente prima e niente dopo, nemmeno `.env.txt`
5. Apri `.env` con il Blocco Note (clic destro → Apri con → Blocco note)
6. Vedrai alcune righe che finiscono con `=`, una per ogni chiave possibile (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GOOGLE_API_KEY`, `FAL_KEY`).  
**Compila solo le righe per cui hai ricevuto una chiave**, incollandola subito dopo il segno `=` (senza spazi, senza virgolette). Le righe per cui non hai ricevuto nulla lasciale vuote così come sono: l'app funziona comunque,

semplicemente quel provider resterà spento finché non gli darai una chiave.

7. Salva (Ctrl+S) e chiudi il Blocco Note.

Questo file resta solo sul tuo computer: non finisce mai su GitHub (è scritto apposta per essere ignorato).

9

## Installare Expo Go sul telefono

L'app, mentre è in sviluppo, non gira come un'app scaricata dal Play Store: gira dentro **Expo Go**, un'app-contenitore gratuita che carica il progetto in tempo reale dal tuo computer. È così che si prova un'app prima che sia pronta per essere pubblicata davvero.

### Perché non lo trovi (ancora) aggiornato sul Play Store

Il progetto usa una versione di Expo (la "57") troppo recente per essere già arrivata sulla versione pubblica di Expo Go sul Play Store — capita spesso con un progetto in sviluppo attivo, non è un problema nostro. La soluzione è scaricare quella versione precisa direttamente dal sito di Expo, la stessa azienda che pubblica l'app sul Play Store.

1. Sul **telefono** (non sul computer), apri il browser e vai su:

```
expo.dev/go?sdkVersion=57&platform=android&device=true
```

2. Nella pagina, tocca il link per scaricare il file `Expo-Go-57.0.2.apk` (o la versione 57.x più recente indicata)
3. Quando il download finisce, tocca il file scaricato per installarlo. Android probabilmente avviserà con qualcosa come "Non è consigliabile installare app da fonti sconosciute" — è il messaggio standard per **qualunque** app scaricata fuori dal Play Store, anche quando è legittima come questa: tocca "Impostazioni", poi attiva "Consenti da questa fonte" per il browser che hai usato, torna indietro e installa.
4. Apri Expo Go una volta, giusto per farla partire. Non serve creare un account né accedere con nessuna email.

### VERIFICA CHE ABBI FUNZIONATO

L'icona di Expo Go compare tra le app del telefono, e aprendola vedi una schermata con un bottone per scansionare un codice QR. La useremo al

DA QUI IN POI: OGNI VOLTA CHE RIAPRI L'APP

10

## Avviare il "cervello" dell'app

Torna alla finestra PowerShell del Passo 6 (quella nella cartella principale `wardrobe`, non dentro `services\api`). Scrivi:

```
npm run api:local
```

Dopo qualche secondo deve apparire una riga tipo:

```
API di Tela in ascolto su http://localhost:8787 (DEV_MODE, dati  
in memoria)
```

### Se Windows chiede il permesso

La prima volta, Windows potrebbe far apparire una finestra **"Windows Defender Firewall ha bloccato alcune funzionalità di questa app"**, per Python. Spunta entrambe le caselle (reti private e reti pubbliche) e clicca **"Consenti l'accesso"**. Senza questo passaggio, l'API funziona sul computer ma il telefono non riuscirà a raggiungerla al Passo 11 — è quasi sempre questa la causa quando l'app resta bloccata dopo aver scattato una foto.

**Importante:** lascia questa finestra aperta. Se la chiudi, l'app smette di funzionare. Puoi ridurla a icona, ma non chiuderla finché la stai usando.

11

## Avviare l'app sul telefono

Controlla prima che **telefono e computer siano sulla stessa rete Wi-Fi** (stesso router — non funziona se il telefono è sui dati mobili, o se uno dei due ha una VPN attiva).

1. Apri una **seconda** finestra PowerShell nella stessa cartella (ripeti il Passo 6: Shift + clic destro nella cartella `wardrobe` → "Apri finestra PowerShell qui")
2. In questa nuova finestra scrivi:



```
npm run mobile
```

3. Aspetta che nel terminale compaia un riquadro fatto di quadratini: è un codice QR.
4. Sul telefono, apri **Expo Go**, tocca "Scan QR code" nella schermata iniziale, e punta la fotocamera sul codice nel terminale.
5. Dopo un po' (la prima volta può metterci un minuto o due) l'app si apre da sola sul telefono.

#### Se Windows chiede di nuovo il permesso

Può apparire un secondo avviso del firewall, questa volta per Node.js: stesso trattamento del Passo 10 – spunta entrambe le caselle e clicca "Consenti l'accesso".

Vuoi solo dare un'occhiata rapida dal computer, senza toccare il telefono? Nella stessa finestra puoi scrivere `npm run mobile:web` invece di `npm run mobile`: si apre una scheda del browser con l'app. Comodo per guardare in fretta, ma la fotocamera vera – e il modo in cui l'app si sentirà per davvero – si provano solo sul telefono.

### Come si usa, in breve

- **Aggiungi** (in basso): scatta una foto o sceglie una dalla galleria di un capo di vestiario. Aspetta che venga letta – vedrai categoria, colore, tessuto proposti. Se qualcosa è scritto su sfondo corallo/arancione, il modello non ne è sicuro: toccalo per correggerlo.
- **Il tuo armadio**: tutti i capi aggiunti.
- **Chiedi a Tela**: scrivi qualcosa tipo *"Fa caldo, sto per uscire, cosa mi metto?"* e aspetta la risposta.

## Provare un provider diverso

Con più di una chiave scritta al Passo 8, puoi scegliere quale modello legge le foto e quale propone gli outfit, senza toccare codice:

1. Nell'app, vai su **Profilo** → **Sviluppo** → **Modelli in uso**
2. Trovi due liste separate — **Lettura foto** e **Suggerimenti** — perché sono i due lavori distinti di cui parlavamo prima del Passo 8, e puoi assegnare un provider diverso a ciascuno
3. **Predefinito del backend** è la prima voce di ogni lista: torna a quello scritto nel `.env`, se mai vuoi annullare la scelta
4. Ogni modello mostra **pronto** o **senza chiave** — è la stessa cosa scritta al Passo 8: senza una riga compilata in `.env`, quel provider resta spento e sceglierlo darebbe un errore

### UNA COSA DA SAPERE

Il `.env` si rilegge solo riavviando la finestra del Passo 10 ( `npm run api:local` ). Se aggiungi o cambi una chiave, chiudi quella finestra e riesegui il comando prima di aspettarti che l'app la veda.

C'è anche una schermata **Playground modelli**, sempre sotto **Sviluppo**: è un banco di misura, non una chat — serve a confrontare un modello con un altro su latenza, costo e qualità della risposta, un test alla volta, senza spostare la scelta stabile delle due liste sopra. Lì vedrai anche `gpt-5.1` e `gemini-2.5-pro`: sono i nomi indicati nel documento di design, non ancora confermati sui listini ufficiali — se un provider risponde con un errore sul nome del modello, è il primo posto da controllare, e si corregge scrivendo `MODELLI_OPENAI` o `MODELLI_GOOGLE` nel `.env`, senza toccare codice.

## Le prossime volte (versione corta)

Non serve rifare i Passi 1-9. Ogni volta che vuoi riaprire l'app:

1. Apri PowerShell nella cartella del progetto (Passo 6), scrivi `npm run api:local`, lascia la finestra aperta

2. Apri una seconda finestra PowerShell nella stessa cartella, scrivi `npm run mobile`, e scansiona il nuovo codice QR con Expo Go (Expo Go tiene anche una cronologia dei progetti recenti in home, ma se il computer ha cambiato indirizzo di rete dall'ultima volta, ripartire dal QR è la strada più sicura)

#### UNA COSA DA SAPERE

Ogni volta che riavvii il punto 1, l'armadio torna quello di partenza – in questa fase di prova i capi non si salvano in modo permanente da un riavvio all'altro. È normale, non è un errore.